

Especificación de Requisitos de Software (SRS) — BIOPET

Índice

1.1. Propósito	2
1.2. Alcance	2
1.3. Definiciones, acrónimos y abreviaturas	2
1.4. Referencias	3
1.5. Resumen del documento	3
2. Descripción global	3
2.1. Perspectiva del producto	3
2.2. Cambio de plataforma tecnológica respecto a la Entrega 1A	3
2.3. Funciones del producto (resumen)	3
2.4. Características de los usuarios	3
2.5. Restricciones	4
2.6. Supuestos y dependencias	4
3. Requisitos específicos	4
3.1. Requisitos funcionales (REQ-F)	4
3.2. Requisitos no funcionales (REQ-NF)	9
4. Trazabilidad (resumen)	11
4.1. Trazabilidad histórica: identificadores originales → identificadores actuales (cierre de OBS-03)	11
5. Modelo de datos (referencia)	13
6. Interfaces de usuario (referencia)	14

Versión: v0.9.0-rc (Tercera Entrega, PFC Aplicaciones Web 2026-2027) **Conforme a:** ISO/IEC/IEEE 29148:2018 (estructura de SRS), INCOSE Guide to Writing Requirements v4 (calidad de requisitos individuales y de conjunto), criterios INVEST de Cohn (historias de usuario), plantilla de Cockburn (casos de uso).

Procedencia de este documento: este archivo se reconstruye a partir de dos fuentes: (1) el SRS de la Entrega 1A (PFC_Entrega1A_BMT.pdf), que especificaba el backend sobre ASP.NET Core 8/C#, y (2) el código realmente implementado en las Entregas 1B y Tercera Entrega, sobre **Java 21 + Spring Boot 3.2 + Spring Security 6 + Spring Data JPA + PostgreSQL 16 + Redis 7**. Ningún requisito de este documento fue inventado sin sustento: cada uno proviene del SRS original, del código fuente verificado, o de la Guía oficial de la Tercera Entrega. El detalle de cada cambio respecto al documento original está en docs/requisitos/cambios/CAMBIOS-SRS.md.

1.1. Propósito

Este documento especifica los requisitos funcionales y no funcionales del sistema BIOPET en su estado v0.9.0-rc (release candidate de la Tercera Entrega del PFC). Su propósito es servir como fuente única de verdad para la matriz de trazabilidad, las pruebas automatizadas y la evidencia empírica exigidas por el bloque A.3 de la Guía de la Tercera Entrega, y como insumo para la Entrega Final (v1.0.0).

1.2. Alcance

BIOPET es un sistema web para centralizar la información clínica y administrativa de clínicas veterinarias de pequeña y mediana escala. El alcance completo del producto (definido en la Entrega 1A) contempla gestión de dueños y mascotas, historial clínico, citas, telemetría IoT, recomendación clínica asistida, facturación digital y reportes.

Alcance implementado y verificado en v0.9.0-rc: autenticación completa (registro, login, refresh, logout con revocación), control de acceso por rol (RBAC), CRUD completo de la entidad Mascota con verificación de propiedad, y resumen agregado de mascotas por especie (vía función SQL). Este es el subconjunto Must Have que el equipo se comprometió a entregar operativo y reproducible en este semestre.

Alcance pendiente, heredado de la Entrega 1A: historial clínico, prescripción de medicamentos, citas veterinarias, telemetría IoT, ubicación en mapa, recomendaciones asistidas, facturación digital y reportes exportables. El modelo de datos conceptual de estos módulos ya existe (ver sección 5 y el DER de la Entrega 1A), pero no hay código de backend ni de frontend implementado todavía. Se documentan como requisitos con estado “pendiente” para que la matriz de trazabilidad (bloque A.3.3 de la Guía) los declare correctamente, en vez de omitirlos.

1.3. Definiciones, acrónimos y abreviaturas

Término	Definición
RBAC	Role-Based Access Control: control de acceso basado en el rol del usuario autenticado.
JWT	JSON Web Token (RFC 7519): token firmado que representa afirmaciones sobre un sujeto autenticado.
ORM	Object-Relational Mapping: mapeo objeto-relacional (Spring Data JPA / Hibernate en este proyecto).
SP	Stored Procedure / función almacenada en el motor de base de datos.
TTL	Time To Live: tiempo de vida de una entrada de caché.
MoSCoW	Técnica de priorización: Must, Should, Could, Won't.

Término	Definición
shall	Verbo modal normativo de ISO/IEC/IEEE 29148 que indica obligación contractual del requisito.

1.4. Referencias

- ISO/IEC/IEEE 29148:2018 — Requirements Engineering.
- INCOSE Guide to Writing Requirements v4.
- RFC 7519 (JWT), RFC 7807 (ProblemDetails).
- Guía de la Tercera Entrega — PFC Aplicaciones Web 2026-2027, UTEQ.
- PFC_Entrega1A_BMT.pdf — SRS y diseño original (ASP.NET Core).
- docs/requisitos/cambios/CAMBIOS-SRS.md — registro de cambios respecto al documento original.

1.5. Resumen del documento

La sección 2 describe el producto de forma global (perspectiva, funciones, usuarios, restricciones). La sección 3 contiene el detalle de cada requisito funcional y no funcional con su identificador persistente, prioridad MoSCoW, criterio de aceptación y método de verificación. La sección 4 resume la trazabilidad end-to-end. Las secciones 5 y 6 remiten a los artefactos de modelo de datos e interfaz que ya existen en el repositorio. La sección 7 documenta, sin inventar contenido, lo que aún falta por completar.

2. Descripción global

2.1. Perspectiva del producto

BIOPET es un sistema nuevo, no una extensión de un producto previo existente en la clínica. Reemplaza procesos manuales (hojas de cálculo, mensajería) por una plataforma web centralizada, según el problema documentado en la Entrega 1A (entrevistas a tres veterinarios y dos auxiliares, encuestas a quince dueños de mascotas).

2.2. Cambio de plataforma tecnológica respecto a la Entrega 1A

El SRS original (Entrega 1A) especificaba ASP.NET Core 8/C# como backend (ADR-001 original) y Bootstrap 5 + HTML/CSS/JS como frontend. El equipo migró la implementación real a **Java 21** + **Spring Boot 3.2** en el backend y **Angular 17+** en el frontend, decisión reflejada en ADR-002-pila-tecnologica.md del repositorio actual. Los requisitos funcionales de negocio (qué hace el sistema) no cambiaron; los requisitos no funcionales técnicos (cómo se implementa: framework, ORM, mecanismo JWT) se actualizaron para reflejar la pila real, evitando que el SRS describa un sistema que no es el que existe en el repositorio.

2.3. Funciones del producto (resumen)

- Gestión de usuarios y autenticación (roles: ADMIN, VETERINARIO, AUXILIAR, DUENO).
- Gestión de mascotas (CRUD + resumen agregado por especie).
- (*Pendiente*) Gestión de historial clínico, citas, telemetría IoT, recomendaciones asistidas, facturación y reportes.

2.4. Características de los usuarios

Rol	Descripción	Nivel técnico esperado
ADMIN	Supervisa roles, operatividad general del sistema y tiene visibilidad global de datos.	Medio-alto (personal administrativo de la clínica).

Rol	Descripción	Nivel técnico esperado
VETERINARIO	Registra diagnósticos y gestiona historiales clínicos (módulo pendiente); gestiona mascotas.	Medio (personal clínico).
AUXILIAR	Apoya operaciones administrativas y de gestión de mascotas.	Medio.
DUENO	Consulta únicamente la información de sus propias mascotas.	Básico (público general, sin capacitación previa).

2.5. Restricciones

- El proyecto debe completarse dentro de un semestre académico (PPA 2026-2027), lo que limita el alcance implementado al núcleo Must Have.
- La base de datos debe ser PostgreSQL 16 (decisión ya tomada, ver ADR-004-postgresql.md).
- La autenticación debe usar JWT en cookies `HttpOnly + Secure + SameSite=Strict` (bloque A.1 de la Guía), no `localStorage` ni `sessionStorage`.
- Toda operación de base de datos que no sea CRUD elemental debe implementarse como función/procedimiento almacenado (bloque A.2 de la Guía), no como JPQL/HQL con joins o agregaciones.

2.6. Supuestos y dependencias

- Se asume disponibilidad de un motor PostgreSQL 16 y Redis 7 accesibles desde el backend (vía Docker Compose en desarrollo).
- Se asume que los servicios externos mencionados en la Entrega 1A (IoT, IA cognitiva, correo) se integrarán en fases posteriores; ninguno está implementado todavía y no se documentan requisitos técnicos de integración hasta que exista una decisión de arquitectura formal para ellos.

3. Requisitos específicos

Cada requisito seguirá el patrón [condición] [sujeto] shall [acción] [objeto] [restricción] de ISO/IEC/IEEE 29148, con identificador persistente, rationale, prioridad MoSCoW, criterio de aceptación medible y método de verificación, cumpliendo las características INCOSE C1–C9 (Necessary, Appropriate, Unambiguous, Complete, Singular, Feasible, Verifiable, Correct, Conforming).

3.1. Requisitos funcionales (REQ-F)

Nota de consistencia (corrección respecto a una versión anterior de este documento):

la numeración de esta sección es la que ya está fijada en `docs/requisitos/historias/HistoriasUsuario.md` (HU-001 a HU-020) y `docs/requisitos/casos-de-uso/CasosDeUso.md` (CU-01 a CU-20), documentos que ya existían en el repositorio con 20 requisitos funcionales completos (incluyendo uno propio para “consultar mascota por id” y otro para “auditoría”, que una reconstrucción anterior de este SRS no tenía desglosados igual). Esta versión del SRS adopta esa numeración como única fuente de verdad, para que los tres documentos no se contradigan entre sí.

Implementados y verificados (REQ-F-001 a REQ-F-012, REQ-F-021) **REQ-F-001 — Registro de usuario dueño de mascota** - **Tipo:** Funcional · **Prioridad:** Must - **Enunciado:** El sistema deberá permitir que cualquier visitante se registre proporcionando nombre, correo electrónico y contraseña, asignándole automáticamente el rol `ROLE_DUENO`, ignorando cualquier rol enviado por el cliente. - **Rationale:** heredado de RF-01 de la Entrega 1A; el registro público solo aplica al rol dueño, los demás roles se crean

administrativamente. - **Verificación:** Test AuthControllerTest (registro exitoso). - **Trazabilidad:** → HU-001 → CU-01 → AuthController.registro → AuthService.registrar → POST /api/auth/registro. - **Estado:** verificado.

REQ-F-002 — Rechazo de registro con correo duplicado - **Tipo:** Funcional · **Prioridad:** Must - **Enunciado:** Al recibir una solicitud de registro con un correo electrónico ya existente, el sistema deberá rechazarla con un código 409 Conflict y un cuerpo ProblemDetails, sin crear ningún registro. - **Rationale:** funcionalidad presente en el código (EmailDuplicadoException) no documentada como requisito independiente en el SRS original de la Entrega 1A; se agrega para cerrar el hueco (bloque A.3 de la Guía). - **Verificación:** Test AuthControllerTest (registro con correo duplicado). - **Trazabilidad:** → HU-001 → CU-01 → AuthService.registrar → EmailDuplicadoException → GlobalExceptionHandler. - **Estado:** verificado.

REQ-F-003 — Autenticación mediante usuario y contraseña - **Tipo:** Funcional · **Prioridad:** Must - **Enunciado:** El sistema deberá permitir que un usuario registrado inicie sesión mediante correo electrónico y contraseña, emitiendo un access token (1 hora de vigencia) y un refresh token con los siete claims JWT estándar. - **Rationale:** heredado de RF-16/RF-WEB-01 de la Entrega 1A. - **Verificación:** Test AuthControllerTest, JwtServiceTest. - **Trazabilidad:** → HU-002 → CU-02 → AuthController.login → POST /api/auth/login. - **Estado:** verificado.

REQ-F-004 — Renovación de sesión (refresh) - **Tipo:** Funcional · **Prioridad:** Must - **Enunciado:** El sistema deberá permitir renovar el access token utilizando un refresh token válido y no revocado, sin exigir nuevamente las credenciales del usuario. - **Rationale:** heredado de RNF-03/RNF-WEB-03 de la Entrega 1A (expiración configurable de JWT), llevado a requisito funcional explícito. - **Verificación:** Test AuthControllerTest (flujo refresh). - **Trazabilidad:** → HU-003 → CU-03 → AuthController.refresh → POST /api/auth/refresh. - **Estado:** verificado.

REQ-F-005 — Cierre de sesión con revocación de token - **Tipo:** Funcional · **Prioridad:** Must - **Enunciado:** El sistema deberá permitir cerrar sesión, registrando el jti del token vigente en una lista negra en Redis con TTL igual a su tiempo restante de expiración, de modo que una solicitud posterior con ese token a un recurso protegido responda 401. - **Rationale:** heredado de RF-17/RF-WEB-04 de la Entrega 1A; mecanismo de revocación definido en ADR-003-jwt-redis.md. - **Verificación:** Test AuthControllerTest, AuthenticationAuditServiceTest. - **Trazabilidad:** → HU-004 → CU-04 → AuthController.logout → TokenBlacklistService.revoke → POST /api/auth/logout. - **Estado:** verificado.

REQ-F-006 — Control de acceso por rol (RBAC) - **Tipo:** Funcional · **Prioridad:** Must - **Enunciado:** El sistema deberá restringir el acceso a cada endpoint protegido según el rol del usuario autenticado, rechazando con 403 Forbidden cualquier solicitud de un rol no autorizado para ese recurso. - **Rationale:** heredado de RF-13/RF-WEB-02 de la Entrega 1A; caso de uso transversal incluido implícitamente por la gestión de mascotas. - **Verificación:** Test MascotaControllerTest (casos 403 por rol). - **Trazabilidad:** → HU-005 → CU-05 → anotaciones @PreAuthorize en MascotaController. - **Estado:** verificado.

REQ-F-007 — Consulta del perfil propio - **Tipo:** Funcional · **Prioridad:** Should - **Enunciado:** El sistema deberá permitir que un usuario autenticado consulte sus propios datos de perfil (nombre, correo, rol), sin exponer el listado completo de usuarios. - **Rationale:** funcionalidad presente en el código (GET /api/usuarios/me), base del authGuard de Angular; no documentada como requisito independiente en el SRS original. - **Verificación:** inspección de UsuarioController y prueba manual con Postman (sin test automatizado formal registrado). - **Trazabilidad:** → HU-006 → CU-06 → UsuarioController.me → AuthService.perfil. - **Estado:** verificado.

REQ-F-008 — Creación de mascota - **Tipo:** Funcional · **Prioridad:** Must - **Enunciado:** El sistema deberá permitir que un usuario con rol ADMIN, VETERINARIO o AUXILIAR registre una nueva mascota (nombre, especie, raza, fecha de nacimiento) asociada a un dueño existente con rol ROLE_DUEÑO. - **Rationale:** heredado de RF-01/RF-02 de la Entrega 1A (registro y asociación con propietario), acotado a la entidad Mascota implementada en esta fase. - **Verificación:** Test MascotaControllerTest (creación exitosa y validación de campos). - **Trazabilidad:** → HU-007 → CU-07 → MascotaController.crear → POST /api/mascotas. - **Estado:** verificado.

REQ-F-009 — Listado paginado de mascotas activas, con filtrado por propietario según rol - **Tipo:** Funcional · **Prioridad:** Must - **Enunciado:** El sistema deberá permitir consultar el listado paginado de mascotas activas; si el solicitante tiene rol `ROLE_DUENO`, el listado deberá restringirse únicamente a sus propias mascotas; para los demás roles autorizados, el listado deberá incluir todas las mascotas activas. - **Rationale:** heredado de RF-02 de la Entrega 1A, con la regla de aislamiento de datos entre dueños explícita en el código (`MascotaService.listar`). - **Verificación:** `Test MascotaControllerTest` (listado por rol `ROLE_DUENO` vs. resto). - **Trazabilidad:** → HU-008 → CU-08 → `MascotaController.listar` → GET `/api/mascotas`. - **Estado:** verificado.

REQ-F-010 — Consulta de mascota por identificador - **Tipo:** Funcional · **Prioridad:** Must - **Enunciado:** El sistema deberá permitir consultar el detalle completo de una mascota a partir de su identificador, respondiendo 404 con `ProblemDetails` si no existe. - **Rationale:** complementa REQ-F-009 para el caso de consulta puntual, necesario antes de una edición o de mostrar el detalle en la interfaz; no estaba desglosado como requisito independiente en una versión anterior de este documento. - **Verificación:** `Test MascotaControllerTest` y `RecursoNoEncontradoException`. - **Trazabilidad:** → HU-009 → CU-09 → `MascotaController.buscar` → GET `/api/mascotas/{id}`. - **Estado:** verificado.

REQ-F-011 — Actualización de mascota con verificación de propiedad - **Tipo:** Funcional · **Prioridad:** Must - **Enunciado:** El sistema deberá permitir actualizar los datos de una mascota existente a un usuario con rol `ADMIN`, `VETERINARIO` o `AUXILIAR`, registrando la fecha de actualización (`actualizado_en`). - **Rationale:** heredado de RF-02 de la Entrega 1A. - **Verificación:** `Test MascotaControllerTest` (actualización exitosa y caso 403/404). - **Trazabilidad:** → HU-010 → CU-10 → `MascotaController.actualizar` → PUT `/api/mascotas/{id}`. - **Estado:** verificado.

REQ-F-012 — Baja lógica de una mascota - **Tipo:** Funcional · **Prioridad:** Must - **Enunciado:** El sistema deberá permitir dar de baja lógicamente (cambio del atributo `activo` a `false`) una mascota existente, sin eliminar físicamente el registro de la base de datos. - **Rationale:** preservación de historial e integridad referencial futura con historial clínico y citas (módulos pendientes). - **Verificación:** `Test MascotaControllerTest` (baja y verificación de que la mascota deja de aparecer en el listado activo). - **Trazabilidad:** → HU-011 → CU-11 → `MascotaController.eliminar` → DELETE `/api/mascotas/{id}`. - **Estado:** verificado.

REQ-F-021 — Resumen de mascotas activas agrupadas por especie - **Tipo:** Funcional · **Prioridad:** Should - **Enunciado:** El sistema deberá permitir consultar el total de mascotas activas agrupadas por especie; para roles distintos de `ADMIN`, el resultado deberá restringirse siempre a las mascotas del propio usuario autenticado; para el rol `ADMIN`, el resultado deberá poder filtrarse por un dueño específico o consultarse sin filtro para obtener el total global. - **Rationale:** nuevo requisito de la Tercera Entrega, exigido por el bloque A.2.2 de la Guía como ejemplo de operación agregada (`GROUP BY`) que debe implementarse obligatoriamente vía función/procedimiento almacenado, no vía JPQL. Se numera 021 (no 013) porque los identificadores 013 a 020 ya estaban ocupados por los requisitos heredados de la Entrega 1A, fijados en `HistoriasUsuario.md/CasosDeUso.md` antes de que este requisito existiera. - **Verificación:** Test de integración `ResumenEspeciesIntegrationTest` (Testcontainers, PostgreSQL real). - **Trazabilidad:** → HU-020 → CU-20 → `MascotaController.resumenPorEspecies` → `MascotaService.resumenPorEspecie` → `MascotaRepository.resumenPorEspecie` → función `fn_resumen_mascotas_por_especie` → GET `/api/mascotas/resumen-especies`. - **Tipo de acceso a datos:** SP (agregación con `GROUP BY`, prohibido en ORM elemental por el bloque A.2.1 de la Guía). - **Estado:** verificado.

Pendientes, heredados de la Entrega 1A (REQ-F-013 a REQ-F-020)

Estos requisitos provienen directamente de RF-03 a RF-15 del SRS original y están desglosados exactamente como en `HistoriasUsuario.md` (HU-012 a HU-019) y `CasosDeUso.md` (CU-12 a CU-19). El modelo de datos conceptual ya existe en el DER de la Entrega 1A (entidades `Cita`, `Historial_Clinico`, `Dispositivo_IoT`, `Chat_Triage`, `Producto_Servicio`, `Factura`, etc.), pero **no hay código de backend ni de frontend implementado** para ninguno de ellos en v0.9.0-rc. Se reclasifican de prioridad Must/Media (documento original) a Should/Could, porque no forman parte del compromiso de esta entrega, y se marcan como “pendiente” para que la matriz de trazabilidad los declare sin ocultarlos.

Id	Descripción (resumen)	Prioridad	Origen	HU / CU	Estado
REQ-F-013	Registrar atención médica y almacenar/consultar el historial clínico de cada mascota de forma cronológica.	Should	RF-03, RF-04	HU-012 / CU-12	pendiente
REQ-F-014	Registrar medicamentos prescritos durante una atención médica.	Could	RF-05	HU-013 / CU-13	pendiente
REQ-F-015	Registrar, modificar y consultar citas veterinarias mediante calendario interactivo.	Should	RF-06	HU-014 / CU-14	pendiente
REQ-F-016	Proveer una API para recibir datos de dispositivos IoT de rastreo asociados a mascotas.	Could	RF-08, RF-09	HU-015 / CU-15	pendiente
REQ-F-017	Generar recomendaciones clínicas informativas a partir del historial médico (enunciado ampliado — ver bloque detallado, cierre de OBS-04).	Could	RF-10	HU-016 / CU-16	pendiente
REQ-F-018	Generar comprobantes de pago digitales en PDF y registrar el método de pago de un servicio veterinario.	Should	RF-11, RF-12	HU-017 / CU-17	pendiente

Id	Descripción (resumen)	Prioridad	Origen	HU / CU	Estado
REQ-F-019	Generar reportes estadísticos exportables en PDF y Excel.	Could	RF-14	HU-018 / CU-18	pendiente
REQ-F-020	Registrar las operaciones relevantes del sistema (usuario, fecha, hora) para fines de auditoría.	Should	RF-15	HU-019 / CU-19	parcial
REQ-F-022	Enviar notificaciones al usuario por correo electrónico ante eventos relevantes de su cuenta.	Could	RF-07	HU-021 / CU-21	pendiente

Nota sobre REQ-F-020 (auditoría): a diferencia de los demás requisitos de esta tabla, no está 100% pendiente: el registro de eventos de *autenticación* (login exitoso/fallido, bloqueo por rate limit, refresh, logout) ya opera en producción y está cubierto por REQ-NF-009 (*AuthenticationAuditService*). Lo que falta es la auditoría genérica de operaciones CRUD sobre mascotas y los módulos pendientes. Se marca “parcial”, no “pendiente”, para no ocultar el trabajo ya hecho.

Nota sobre REQ-F-017 (cierre de OBS-04): la retroalimentación oficial del SGA de la Entrega 1A señaló, como observación leve, la ambigüedad de la redacción original de RF-10 (“recomendaciones informativas”; ver `docs/observaciones/OBSERVACIONES.md`, OBS-04). El identificador y el alcance no cambian (sigue siendo REQ-F-017, prioridad *Could*, estado *pendiente*, dependiente de REQ-F-013); lo que cambia es exclusivamente la redacción, reformulada en patrón “El sistema deberá...” con entradas, resultado esperado y criterios de aceptación verificables, sin ampliar la funcionalidad prevista originalmente. El detalle completo reemplaza al resumen de una sola línea que tenía esta fila en versiones anteriores del SRS.

REQ-F-017 — Generación de recomendaciones clínicas informativas a partir del historial médico - Tipo: Funcional · **Prioridad:** Could · **Enunciado:** Al recibir una solicitud de recomendaciones para una mascota con historial clínico registrado, el sistema deberá generar, mediante un servicio de IA externo, una lista de recomendaciones de cuidado en texto a partir de los datos del historial clínico de esa mascota, y deberá devolver cada recomendación acompañada de la advertencia explícita “informativa, no sustituye diagnóstico veterinario”. · **Entradas:** identificador de una mascota con historial clínico registrado (depende de REQ-F-013, módulo pendiente). · **Resultado esperado:** una lista de cero a N recomendaciones en texto plano, cada una acompañada de la advertencia de carácter informativo; si la mascota no tiene historial clínico registrado, el sistema deberá responder con una lista vacía, no con un error. · **Criterios de aceptación (verificables):** 1. Dado un historial clínico existente para la mascota solicitada, cuando se invoca el endpoint de recomendaciones, entonces la respuesta incluye el campo `recomendaciones: string[]` y, por cada elemento, el campo `advertencia: "informativa, no sustituye diagnóstico veterinario"`. 2. Dado que la mascota no tiene historial clínico registrado, cuando se solicita el endpoint de recomendaciones, entonces la respuesta es 200 OK con `recomendaciones: []`. 3. El proveedor concreto del servicio de IA es una decisión de arquitectura pendiente; no se implementa en v0.9.0-rc. · **Rationale:** heredado de RF-10 de la

Entrega 1A; reformulado en la Tercera Entrega para cerrar OBS-04 (ambigüedad leve señalada por el docente en “recomendaciones informativas”), sin ampliar el alcance original: sigue dependiendo de REQ-F-013 (historial clínico, módulo pendiente) y del mismo servicio de IA externo ya previsto desde la Entrega 1A. - **Verificación:** pendiente — no implementado en v0.9.0-rc (depende de REQ-F-013). Método de verificación previsto: test de integración con un doble de prueba (mock) del servicio de IA, validando el contrato de entrada/salida y el caso sin historial clínico. - **Trazabilidad:** → HU-016 → CU-16 → (módulo pendiente; depende de REQ-F-013). - **Estado:** pendiente (redacción cerrada; implementación no iniciada).

Nota sobre REQ-F-022 (cierre de OBS-02): este requisito no forma parte de la secuencia original RF-03 a RF-15; se numera 022 (siguiente identificador libre después de REQ-F-021) porque corresponde a **RF-07**, el requisito que la retroalimentación oficial del SGA de la Entrega 1A señaló como ausente de la lista consolidada (“FALTA RF-07 en la lista consolidada, salta RF-06 -> RF-08”; ver docs/observaciones/OBSERVACIONES.md, OBS-02). RF-07 no estaba perdido: el diagrama de contexto C4 Nivel 1 (docs/diagrams/c4-contexto/C4-L1-contexto.md, sección “Trazabilidad”) ya lo identificaba como el “Servicio de Correos” mencionado en la Entrega 1A, sin que ninguna versión anterior de este SRS le hubiera asignado un identificador REQ-F propio. Se le asigna aquí REQ-F-022 (no REQ-F-007, que ya está ocupado por un requisito distinto — “Consulta del perfil propio” — para no duplicar identificadores) con su enunciado completo, criterios de aceptación verificables y trazabilidad propia (ver bloque detallado más abajo).

REQ-F-022 — Notificaciones al usuario por correo electrónico (Servicio de Correos, RF-07 recuperado) - Tipo: Funcional · **Prioridad:** Could - **Enunciado:** Al ocurrir un evento relevante para la cuenta de un usuario (por ejemplo, registro exitoso), el sistema deberá enviar una notificación por correo electrónico al usuario afectado a través de un servicio de correo externo, de forma asíncrona, sin bloquear ni condicionar la respuesta HTTP de la operación que originó el evento. - **Entradas:** un evento de dominio notificable (tipo de evento, correo electrónico destinatario, datos contextuales mínimos del evento). - **Resultado esperado:** el correo se envía o se encola para envío de forma asíncrona; un fallo del servicio de correo externo no debe revertir ni hacer fallar la operación de dominio que originó el evento. - **Criterios de aceptación (verificables):** 1. Dado un registro de usuario exitoso, cuando el servicio de correo está disponible, entonces se envía un correo de bienvenida a la dirección registrada. 2. Dado que el servicio de correo externo no está disponible, cuando ocurre un evento notificable, entonces la operación de dominio que lo originó (por ejemplo, el registro) completa exitosamente y el fallo de envío queda registrado en el log del sistema, sin propagarse como error al cliente. 3. El proveedor de correo concreto (SMTP propio, SES, SendGrid u otro) es una decisión de arquitectura pendiente; no está implementado en v0.9.0-rc. - **Rationale:** corresponde al RF-07 original de la Entrega 1A (“Servicio de Correos”), documentado como sistema externo en docs/diagrams/c4-contexto/C4-L1-contexto.md pero sin requisito REQ-F formal hasta esta revisión. Se incorpora explícitamente para cerrar OBS-02 sin duplicar REQ-F-007 (funcionalidad distinta ya implementada). - **Verificación:** pendiente — no implementado en v0.9.0-rc. Método de verificación previsto: test de integración con un servidor SMTP de pruebas (por ejemplo, Mailhog) que confirme el envío en el caso exitoso y el comportamiento no bloqueante ante fallo del servicio externo. - **Trazabilidad:** → HU-021 → CU-21 → (módulo pendiente; sin controlador ni servicio implementados en el backend). - **Estado:** pendiente.

3.2. Requisitos no funcionales (REQ-NF)

REQ-NF-001 — Rendimiento del listado de mascotas - Categoría: Rendimiento · **Prioridad:** Must - **Enunciado:** El tiempo de respuesta del listado de mascotas no deberá superar 200 ms (p95) con caché caliente ni 500 ms (p95) con caché fría. - **Rationale:** heredado de RNF-01/RNF-WEB-04 de la Entrega 1A, con umbrales cuantitativos añadidos por el bloque C.1 de la Guía. - **Verificación:** k6 (50 VUs, 30 s), 3 corridas, docs/mediciones/perf/. - **Estado:** pendiente de evidencia empírica archivada (bloque C.1).

REQ-NF-002 — Comunicación cifrada obligatoria - Categoría: Seguridad · **Prioridad:** Must - **Enunciado:** Toda comunicación entre cliente y servidor deberá realizarse mediante HTTPS/TLS. - **Rationale:** heredado de RNF-02/RNF-WEB-02. - **Verificación:** curl -v mostrando TLSv1.3 y suite AEAD; SecurityHeadersTest. - **Trazabilidad:** docker-compose.tls.yml, application-tls.yml, scripts/generate-dev-keystore.sh. - **Estado:** verificado en configuración TLS de desarrollo.

REQ-NF-003 — Expiración configurable de tokens JWT - Categoría: Seguridad · **Prioridad:** Must - **Enunciado:** El sistema deberá emitir access tokens con expiración de 1 hora y refresh tokens con expiración de 7 días, ambos configurables mediante variables de entorno (JWT_EXPIRATION_MS, JWT_REFRESH_EXPIRATION_MS), sin valores fijos en el código. - **Rationale:** heredado de RNF-03/RNF-WEB-03. - **Verificación:** JwtServiceTest; inspección de application.yml. - **Estado:** verificado.

REQ-NF-004 — Claims estándar del JWT - Categoría: Seguridad · **Prioridad:** Must - **Enunciado:** Cada JWT emitido deberá incluir los siete claims estándar (iss, sub, aud, exp, nbf, iat, jti) conforme al RFC 7519. - **Rationale:** refinamiento de RNF-03 exigido por el bloque A.1 de la Guía; verificado directamente en JwtService.java (issuer, audience configurables vía JWT_ISSUER/JWT_AUDIENCE). - **Verificación:** JwtServiceTest. - **Estado:** verificado.

REQ-NF-005 — Interfaz responsiva - Categoría: Usabilidad · **Prioridad:** Must - **Enunciado:** La interfaz deberá adaptarse correctamente a resoluciones entre 320px y 1440px (móvil, tablet, escritorio). - **Rationale:** heredado de RNF-04/RNF-WEB-01. - **Verificación:** Lighthouse (perfil móvil, Slow 4G) + inspección manual en Chrome DevTools a 320px, 768px y 1440px. - **Trazabilidad:** frontend/src/app/features/mascotas.component.ts (.grid-mascotas con @media (max-width: 600px)). - **Estado:** verificado parcialmente (pantalla de Mascotas); pendiente de auditoría Lighthouse archivada.

REQ-NF-006 — Compatibilidad de navegadores - Categoría: Compatibilidad · **Prioridad:** Should - **Enunciado:** La aplicación deberá funcionar correctamente en las versiones modernas de Chrome, Firefox y Edge. - **Rationale:** heredado de RNF-05/RNF-WEB-05. - **Verificación:** ejecución manual de los flujos críticos en los tres navegadores. - **Estado:** pendiente de evidencia archivada.

REQ-NF-007 — Disponibilidad durante evaluaciones académicas - Categoría: Disponibilidad · **Prioridad:** Must - **Enunciado:** El sistema deberá estar operativo durante las semanas de evaluación establecidas por la asignatura. - **Rationale:** heredado de RNF-06. - **Verificación:** demostración en vivo (make up) durante la semana de entrega. - **Estado:** verificado por diseño (reproducibilidad automática, bloque B).

REQ-NF-008 — Cifrado de contraseñas - Categoría: Seguridad · **Prioridad:** Must - **Enunciado:** Las contraseñas de los usuarios deberán almacenarse cifradas mediante un algoritmo hash seguro (BCrypt), nunca en texto plano. - **Rationale:** heredado de RNF-07. - **Verificación:** AuthControllerTest; inspección de db/seed.sql (hash BCrypt del usuario admin) y de SecurityConfig (PasswordEncoder). - **Estado:** verificado.

REQ-NF-009 — Registro de eventos de autenticación (OWASP A09) - Categoría: Seguridad · **Prioridad:** Must - **Enunciado:** El sistema deberá registrar cada evento de autenticación (login exitoso, login fallido, bloqueo por rate limit, refresh, logout) con IP, marca de tiempo y sujeto (correo), sin exceder 200 caracteres por campo registrado. - **Rationale:** requisito nuevo derivado del control OWASP A09, exigido por el bloque C.2 de la Guía; ausente en el SRS original. Verificado directamente en AuthenticationAuditService.java. - **Verificación:** AuthenticationAuditServiceTest; captura de log real para el bloque C.2 (docs/mediciones/sec/). - **Estado:** verificado en código; pendiente de evidencia archivada (captura de log de producción/staging).

REQ-NF-010 — Limitación de intentos de login (OWASP A07) - Categoría: Seguridad · **Prioridad:** Must - **Enunciado:** El sistema deberá bloquear temporalmente los intentos de login desde una misma IP tras 6 intentos fallidos consecutivos dentro de una ventana de 15 minutos, respondiendo 429 Too Many Requests durante el bloqueo, con parámetros configurables mediante variables de entorno. - **Rationale:** requisito nuevo derivado del control OWASP A07, exigido por el bloque C.2 de la Guía; ausente en el SRS original. Verificado directamente en LoginRateLimiterService.java (security.rate-limit.login.max-attempts, .window, .block-duration). - **Verificación:** LoginRateLimiterServiceTest. - **Estado:** verificado en código; pendiente de evidencia curl archivada (bloque C.2, control A07).

REQ-NF-011 — Arquitectura en capas - Categoría: Mantenibilidad · **Prioridad:** Must - **Enunciado:** La arquitectura del backend deberá organizarse en capas separadas de presentación (controller), lógica de negocio (service) y acceso a datos (repository/entity). - **Rationale:** heredado de RNF-08. - **Verificación:** revisión de la estructura de paquetes com.biopet.{controller,service,repository,entity,dto}.

- **Estado:** verificado.

REQ-NF-012 — Caché del listado de mascotas con TTL externo - Categoría: Rendimiento · **Prioridad:** Should - **Enunciado:** El listado paginado de mascotas deberá almacenarse en caché Redis con un tiempo de vida (TTL) configurable mediante variable de entorno (`CACHE_TTL_MS`, valor por defecto 300000 ms), sin valores fijos en código, y su tasa de aciertos (hit ratio) deberá medirse y reportarse empíricamente. - **Rationale:** heredado de la estrategia de caché ya definida en `ADR-003-jwt-redis.md`; llevado a requisito no funcional explícito exigido por el bloque A.1 de la Guía. - **Verificación:** inspección de `application.yml` (`spring.cache.redis.time-to-live`); medición de hit ratio en `docs/mediciones/perf/`. - **Estado:** verificado en configuración; pendiente de evidencia de hit ratio archivada.

REQ-NF-013 — Estrategia híbrida de acceso a datos (ORM + procedimientos almacenados) - Categoría: Mantenibilidad / Seguridad · **Prioridad:** Must - **Enunciado:** Toda operación de base de datos que no sea un CRUD elemental sobre una única tabla (según la definición del bloque A.2.1 de la Guía) deberá implementarse como función o procedimiento almacenado versionado en `db/procs/`, invocado desde un repositorio Spring Data, quedando prohibida la concatenación de entrada de usuario en cualquier fragmento JPQL, HQL o SQL nativo. - **Rationale:** requisito nuevo exigido explícitamente por el bloque A.2 de la Guía. Se marcó “cumplida por alcance actual” en una versión anterior de este documento porque en ese momento no existía ninguna operación no elemental; en v0.9.0-rc ya existe una implementación real: `fn_resumen_mascotas_por_especie` (ver REQ-F-021). - **Verificación:** `ResumenEspeciesIntegrationTest`; `scripts/audit-sql-dynamic.sh` (ausencia de SQL dinámico por concatenación); `docs/basedatos/CATALOGO-SP.md`. - **Estado:** verificado.

4. Trazabilidad (resumen)

La matriz completa vive en `docs/trazabilidad/matriz.csv` (bloque A.3.3 de la Guía), con la fila raíz por requisito y las columnas: `id_requisito`, `tipo`, `prioridad_moscow`, `historia_usuario`, `caso_de_uso`, `modulo_codigo`, `endpoint_api`, `prueba_automatizada`, `tipo_acceso`, `evidencia_empirica`, `estado`. Este SRS es la fuente de verdad para las columnas `id_requisito`, `tipo`, `prioridad_moscow` y `estado`; la matriz no debe declarar un requisito que no exista aquí, ni omitir ninguno de los REQ-F/REQ-NF listados en la sección 3.

Correspondencia REQ-F HU CU (para llenar matriz.csv):

REQ-F	HU	CU	REQ-F	HU	CU
001	HU-001	CU-01	011	HU-010	CU-10
002	HU-001	CU-01	012	HU-011	CU-11
003	HU-002	CU-02	013	HU-012	CU-12
004	HU-003	CU-03	014	HU-013	CU-13
005	HU-004	CU-04	015	HU-014	CU-14
006	HU-005	CU-05	016	HU-015	CU-15
007	HU-006	CU-06	017	HU-016	CU-16
008	HU-007	CU-07	018	HU-017	CU-17
009	HU-008	CU-08	019	HU-018	CU-18
010	HU-009	CU-09	020	HU-019	CU-19
021	HU-020	CU-20	022	HU-021	CU-21

4.1. Trazabilidad histórica: identificadores originales → identificadores actuales (cierre de OBS-03)

La retroalimentación oficial del SGA de la Entrega 1A señaló que “los RF-WEB se remapean a RF-16/RF-17 sin matriz de trazabilidad explícita en esta entrega” (ver `docs/observaciones/OBSERVACIONES.md`, OBS-03). Hasta esta revisión, el vínculo entre los identificadores originales (RF-NN / RF-WEB-NN, Entrega 1A) y los

identificadores actuales (REQ-F-NNN) solo existía disperso en el campo *Rationale* de cada requisito individual (sección 3.1). La siguiente tabla lo consolida en un único lugar explícito, sin omitir ningún requisito funcional del sistema y sin inventar ningún origen que no estuviera ya citado en este documento:

Identificador anterior (Entrega 1A)	Identificador actual	Descripción	Caso de uso	Historia	Estado
RF-01	REQ-F-001	Registro de usuario dueño de mascota	CU-01	HU-001	verificado
RF-01, RF-02	REQ-F-008	Creación de mascota asociada a un dueño existente	CU-07	HU-007	verificado
RF-02	REQ-F-009	Listado paginado de mascotas activas por propietario/rol	CU-08	HU-008	verificado
RF-02	REQ-F-011	Actualización de mascota con verificación de propiedad	CU-10	HU-010	verificado
RF-03, RF-04	REQ-F-013	Registro y consulta de historial clínico	CU-12	HU-012	pendiente
RF-05	REQ-F-014	Prescripción de medicamentos	CU-13	HU-013	pendiente
RF-06	REQ-F-015	Gestión de citas veterinarias mediante calendario	CU-14	HU-014	pendiente
RF-07	REQ-F-022	Notificaciones al usuario por correo electrónico (cierre de OBS-02)	CU-21	HU-021	pendiente
RF-08, RF-09	REQ-F-016	API de recepción de telemetría de dispositivos IoT	CU-15	HU-015	pendiente
RF-10	REQ-F-017	Recomendaciones clínicas informativas (redacción cerrada en OBS-04)	CU-16	HU-016	pendiente
RF-11, RF-12	REQ-F-018	Facturación digital y registro de pagos	CU-17	HU-017	pendiente

Identificador anterior (Entrega 1A)	Identificador actual	Descripción	Caso de uso	Historia	Estado
RF-13, RF-WEB-02	REQ-F-006	Control de acceso por rol (RBAC)	CU-05	HU-005	verificado
RF-14	REQ-F-019	Reportes estadísticos exportables	CU-18	HU-018	pendiente
RF-15	REQ-F-020	Auditoría de operaciones del sistema	CU-19	HU-019	parcial
RF-16, RF-WEB-01	REQ-F-003	Autenticación mediante usuario y contraseña	CU-02	HU-002	verificado
RF-17, RF-WEB-04	REQ-F-005	Cierre de sesión con revocación de token	CU-04	HU-004	verificado
RNF-03, RNF-WEB-03	REQ-F-004	Renovación de sesión (refresh)	CU-03	HU-003	verificado

Requisitos sin origen en la Entrega 1A (nuevos, agregados durante la Tercera Entrega para documentar funcionalidad ya presente en el código, sin requisito previo que cerrar): **REQ-F-002** (rechazo de correo duplicado), **REQ-F-007** (consulta del perfil propio), **REQ-F-010** (consulta de mascota por id), **REQ-F-012** (baja lógica de mascota) y **REQ-F-021** (resumen de mascotas por especie, exigido por el bloque A.2.2 de la Guía de la Tercera Entrega). Ninguno de estos sustituye ni duplica un identificador **RF-NN** original.

Esta tabla es de solo lectura respecto a `docs/trazabilidad/matriz.csv`: no lo reemplaza ni lo modifica (ese archivo está fuera del alcance de los archivos autorizados para este cierre de observaciones). Queda como acción de seguimiento incorporar, en una futura revisión de `matriz.csv`, una columna equivalente de origen histórico para **REQ-F-022**.

5. Modelo de datos (referencia)

El diccionario de datos completo de las entidades implementadas (usuarios, mascotas) está en `docs/diccionario_datos.md`, sincronizado con la migración `Flyway database/migrations/V1__schema_inicial.sql`. El modelo entidad-relación conceptual completo (incluyendo las entidades pendientes: Cita, Historial_Clinico, Dispositivo_IoT, Chat_Triage, Producto_Servicio, Factura, Detalle_Factura, Proveedor, Mercaderia, Detalle_Ingreso) proviene de la Entrega 1A (`PFC_Entrega1A-BMT.pdf`, sección 6) y se conserva como visión de producto pendiente de materialización, sin duplicar aquí el DDL completo.

Diagrama entidad-relación (DER) — dos artefactos distintos, no intercambiables (cierre de OBS-05):

- `docs/diagrams/der-biopet/der-biopet.png` — renderizado generado a partir de la fuente `Graphviz der-biopet.dot`. Es un diagrama **dibujado**, no una exportación de una herramienta de modelado de base de datos.
- `docs/observaciones/evidencias/DER-BIOPET-pgAdmin-ERD-Tool.png` — exportación **real** generada desde **pgAdmin 4**, herramienta **ERD Tool**, solicitada explícitamente por la retroalimentación oficial del SGA de la Entrega 1B: “Exportar el DER desde pgAdmin 4 (ERD Tool) como PNG de alta resolución para el informe final” (ver `docs/observaciones/OBSERVACIONES.md`, OBS-05). PNG válido (firma de archivo verificada), 768×883 px, generado directamente sobre el esquema real de PostgreSQL, no sobre la fuente `.dot`.

6. Interfaces de usuario (referencia)

Los wireframes conceptuales (login unificado, dashboard por rol, historial clínico, gestión de citas) están documentados en la Entrega 1A ([PFC_Entrega1A_BMT.pdf](#), sección 7). La interfaz realmente implementada en v0.9.0-rc (login y gestión de mascotas con paginación, formularios, confirmación de borrado, resumen por especie y accesibilidad) está en `frontend/src/app/features/`, sin wireframe formal actualizado — se deja como observación abierta en la sección 7.